LEAD PACKAGES: AUDIT + COSTED DESIGN, v1

2026-08-31. Prices exactly the ten assumptions in structure v2.0 Part G, against the code as it exists today (main at 0d0cfa1; save schema 1.0.0 and episodes live on `feature/multi-episode-audit`, unmerged). Pattern follows `feature-multi-episode-support-v1.md`: verified facts with line refs, then the bill, then the rulings queue. Nothing here is built.

1. What the code already gives us (verified, with refs)

Part G assumption	Code truth	Verdict
G1 cards are ordinary `LeadData`	`Assets/App/Leads/Data/LeadData.cs`: id, display, `requirements[]`, `RequiredLeadIds`, `SpawnLeadIds`, rewards, `resolutionDialogue`, board fields. Runtime state shadowed in `RuntimeState`, never baked to the SO	**Holds.** Cards need zero schema change
G2 one requirement slot, quantity 1 to 3	`requirements` is an array (max 3 recommended); `LeadRequirement.quantity` is `[Range(1,3)]` (shipped, exercised by Schedule B's L10 fix)	**Holds.** Authoring convention, not code
G5 package-to-package gating with shipped primitives	`LeadsRepository.CheckAndUnlockBlockedLeads` (`LeadsRepository.cs:50`) unlocks a Blocked lead only when **all** `RequiredLeadIds` are in `_activatedLeadIds` (`:65`). AND-gate semantics exist today	**Holds.** Package N+1 member cards each list package N's member ids. No new gate machinery
G7 no new save state	`_activatedLeadIds` round-trips through `LeadSaveState` / `ApplySavedStates` (`LeadsRepository.cs:199-251`) on the existing save path	**Holds, with one exception: the pending-beat marker (§3)**
G4 rewards package-level	`LeadOutcomeMB.GrantRewards` (`LeadOutcomeMB.cs:55-88`) pays per lead via `WalletLocator`, plus specials, generator rewards, flags, spawns	**Holds with a correction (§2):** payout cannot sit on a designated card, because the completing card is whichever the player finishes last
G3 beat fires once, on completion	Per-lead `resolutionDialogue` (`CaseGraph`) is the shipped beat path	Needs the container runtime (§2) and the art+caption surface (§4)
G6 at most ~7 chips	`LeadsBarView.cs` (442 lines) renders flat chips; no grouping concept	UI work item (§5)
G8 ids and assets	Convention only	Free
G9 family availability	Mirrors shipped `e1_tip`-style duties	Editor wiring pass, included in integration
G10 analytics	`LeadAnalytics.cs` exists for lead events	Small addition

The one Part G gap: G4/G3 say "the completing card or the container" fires the beat and pays. The completing card is not knowable at authoring time (G6 lets the player order cards freely inside a package), so a **container runtime must detect completion**. That is the only genuinely new mechanism in the whole feature.

2. The design (chosen, per the no-menus rule)

- `PackageData` **ScriptableObject** (`Assets/App/Leads/Data/`): packageId, ordered member card ids, beat payload (beat type enum; `CaseGraph` ref for dialogue beats; sprite + caption for art beats; character-fact text), package rewards (soft/energy/premium/special), chapter id. A **`PackageCatalog` SO** lists them in order for validation and analytics. Member `LeadData` cards carry **zero rewards and no `resolutionDialogue`**; a one-line toast (§5.7 register) is optional per card.

- **`PackageRuntime` MonoBehaviour** (composition-root wired, like the orchestrator): subscribes to `LeadsRepository.LeadsChanged`; on each change, scans the catalog for packages whose members are all in `ActivatedLeadIds` and whose beat flag is unset (a state-scan, per robustness rule 6, so restore and edge cases converge); fires the beat, pays the rewards through `WalletLocator`, emits analytics, sets the beat flag **after** the beat displays.
- **Gating stays on the cards** (G5 as written): the structure doc's Part B already lists the `RequiredLeadIds` per card. The runtime adds nothing to gating.

3. Robustness compliance (the rules that retired bug classes)

- **Rule 5 (persist done only after it happened):** the package-complete flag (`aq.fk.p01_03.beat_seen` in `GameFlags`) is set only after the beat presentation is dismissed. Crash between the last card's activation and the beat display: the boot-time state-scan re-fires the beat. Rewards must be idempotent with the beat: pay on the same dismiss, or pay first and mark paid separately; **chosen: pay and flag in the same dismiss handler, and accept a re-shown beat as the crash outcome rather than a double payment** (a `beat_paid` flag checked before granting).
- **Rule 1 (save aggregate):** `GameFlags` is already folded into the aggregate on the branch, so beat/paid flags ride the existing atomic save. **No new file, no prefs.** If any per-package state ever exceeds flags, it folds into `BoardSaveSystem` with the standard crash-boundary suite.
- **Rule 6 (state-scan over edge-event):** completion detection is a scan on `LeadsChanged`, never a per-card event chain.
- **Tests (mandatory):** an EditMode suite on the templates' pattern: completion detection with member subsets, beat flag set only after display callback, reward idempotence across simulated crash, catalog validation (every Part B card exists, ids unique, gates acyclic, package totals match the v1.1 envelope), restore-time re-fire.

4. The bill (solo-dev days, after `feature/multi-episode-audit` merges)

#	Item	Days	Notes
1	`PackageData` + `PackageCatalog` + validation	0.5	SO plumbing on existing patterns
2	`PackageRuntime`: scan, beat dispatch, rewards, flags, restore re-fire	1.5	The new mechanism; includes the EditMode suite
3	Beat presentation surface: art+caption panel (new), dialogue beats reuse the shipped `CaseGraph` path, character-fact card variant	1.5 to 2	The only new player-facing UI; portrait/art slot, caption, dismiss
4	Leads-bar grouping: package header chip, member chips, at most 2 packages visible	1 to 2	Heaviest UI risk; `LeadsBarView` is 442 lines and drag/click alternation is a known fragile area, so this gets its own editor QA pass
5	Content tooling: an editor auditor that diffs the built assets against structure v2 Part B (250 cards, ids, gates, T1eq totals, CC bands)	1	At 250 cards, hand-checking is how errors ship; the SO-auditor commitment finally earns its keep

#	Item	Days	Notes
6	FTUE teaching: chapter 1 packages 1 to 3 introduce the concept; copy is Stephen-ruled	0.5 to 1	Rides the existing guided loop
7	Analytics: `package_complete` (chapter, id, T1eq, session index) + climax watches	0.25	On `LeadAnalytics`
8	Family/generator availability wiring (G9) + integration + editor QA	1	Includes the §D chapter-1 seconds check
	Total	**7.25 to 9.25**	**MVP cut: defer 4 to a minimal "package title above its chips" (saves ~1 day) and 5's CC-band check to a spreadsheet diff (saves ~0.5)**

Content authoring (250 `Lead_FK_*` assets + ~100 `PackageData`) is not in this bill; it scales with the Fable-brief and VO pipeline and is the production schedule's line, not a systems line.

5. Sequencing

1. Stephen rules structure v2 Part F; GPT attacks the structure (both can run before any code).
2. `feature/multi-episode-audit`: play-verify + HUD scene pass + **merge**. Packages build on its save schema and flags fold; building them on main first would be double work.
3. Items 1, 2, 7 (the invisible spine), then 3, then 4 and 6, then 5 and 8.

6. Rulings queued for Stephen (none block Part F)

1. **The beat surface's shape:** full-screen interstitial (art fills, caption below) or board-corner card? Cheapest build is the interstitial; the corner card fights the merge board for space.
2. **Member-card toasts:** on 3+ card packages, does completing a non-final card show a §5.7 toast, or nothing? (Structure v2 assumed at most a toast.)
3. **FTUE copy** for teaching the package concept (≤8-word lines, ruled like all copy).
4. **MVP cut yes or no** (item 4's minimal grouping, item 5's spreadsheet diff).